

Understanding an Unfamiliar Codebase with AI: Task Manager Exercise

Submission summary — GenAI for Software Engineers course

1. Initial vs. Final Understanding

Initial: Based only on file names, I guessed Task Manager was a simple task/todo CLI app, with roughly one responsibility per file (cli.py = entry point, storage.py = disk I/O, task_manager.py = “core logic,” task_priority.py = priority levels).

Final: The real architecture is a clean four-layer chain — cli.py (thin argparse dispatcher) → task_manager.py (coordination/validation, not core logic itself) → storage.py (JSON persistence, in-memory dict, custom encoder/decoder for enums and dates) → models.py (Task, TaskStatus, TaskPriority). Critically, three files (task_parser.py, task_priority.py, task_list_merge.py) sit **outside** this chain entirely. task_priority.py in particular contains a fully-built “importance score” feature (calculate_task_score) that is never called from the CLI — a real, undiscovered feature gap rather than dead code, and distinct in meaning from the static TaskPriority enum.

2. Most Valuable Insights Per Prompt

Knowing Where to Start

Following the actual call chain (rather than reading files alphabetically) revealed that task_manager.py is a coordinator, not the “core logic” I assumed — and surfaced three orphaned files worth flagging to a real team.

Finding Feature Implementation Locations

Confirming an absence (no existing export/CSV code) was itself a useful finding: it told me this would be genuinely new work, with storage.py’s JSON-writing pattern as the only reusable precedent — and that new logic should live in its own module to avoid blurring storage.py’s single responsibility.

Understanding Domain Models & Business Concepts

The most valuable prompt overall. Its quiz-style follow-up questions caught an assumption I hadn’t tested — that priority would always dominate the computed importance score. Working the numbers (URGENT/stale = 60 vs. LOW/overdue/blocker = 53) showed the score is a genuine multi-factor trade-off, not a simple priority restatement.

3. Implementing the New Business Rule

Rule: Tasks overdue more than 7 days are auto-marked “abandoned” unless high priority.

Planned changes:

- models.py — add TaskStatus.ABANDONED
- task_manager.py — new apply_abandonment_rule() method, consistent with its existing coordinator role
- storage.py / models.py — unchanged; reuse get_all_tasks() / update_task()
- cli.py — open decision: run automatically on every invocation, or as an explicit subcommand

Open questions for the team:

- Does “high priority” mean HIGH only, or HIGH + URGENT?
- Should the rule run automatically, or only on-demand?
- Is “abandoned” a terminal state, or reversible if the due date changes?
- Does the codebase’s existing lack of timezone handling matter here?

4. Strategies for Approaching Unfamiliar Code Going Forward

- State my current understanding and guesses *before* asking the AI — this is what turns a generic explanation into a targeted correction.
- Follow the actual call chain from the entry point, rather than reading files in isolation or alphabetically.

- Treat a confirmed absence (“no code for X exists”) as a real finding, not a dead end.
- Use quiz-style follow-up questions to test understanding, rather than accepting an explanation at face value — and verify by running the code where possible.
- Keep a running discovery log; it turns one-off exploration into a reusable artifact for onboarding or review.